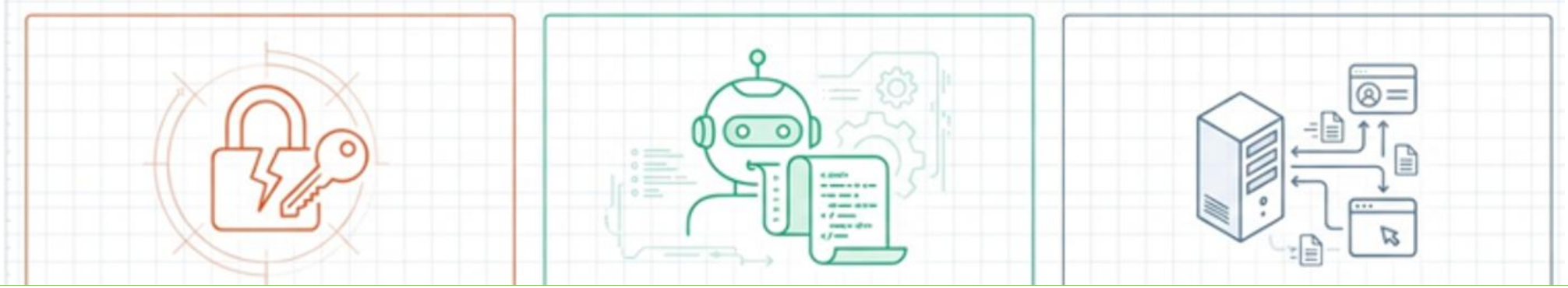
# A07:2025 Authentication Failures

# A07:2025 Authentication Failures

## 漏洞診斷：我們的系統在哪裡失敗？



**憑證與密碼弱點**

- 允許預設或弱密碼（如 'admin/admin'、'Password1'）
- 未檢查已洩漏憑證（Known-breached credentials）
- 脆弱的憑證恢復流程（如 KBA 知識型問題）
- 儲存方式不當（明文、可逆加密或弱雜湊）

**自動化攻擊防禦缺失**

- **Credential Stuffing（憑證填充）**：攻擊者使用洩漏的帳號密碼清單進行撞庫。
- **Brute Force（暴力破解）**：未能及時攔截的腳本化自動攻擊。

**會話與流程漏洞**

- Session ID 暴露於 URL 中
- Session Fixation：登入成功後未更新 Session ID
- 帳號枚舉風險：錯誤訊息透露 'User not found' 或 'Password incorrect'

# A07:2025 Authentication Failures

## 攻擊剖析：代碼如何被利用

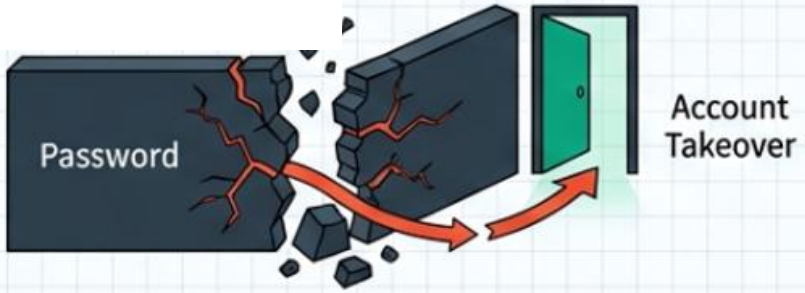
### 混合式憑證填充

攻擊者根據人類行為模式進行猜測。
若系統缺乏自動化防禦，將淪為「**Password Oracle**」，
協助攻擊者驗證憑證有效性。

### 單一登出失效

使用者在 SSO 環境下登出單一服務，
但 Session 在其他子系統仍保持活躍。
攻擊者可在公共終端機上接手未完全中斷的會話。

### 單一因子陷阱




強制密碼複雜度導致使用者重複使用舊密碼。
缺乏 MFA 保護下，單一密碼洩漏即導致帳號完全淪陷。

# A07:2025 Authentication Failures

## 防禦策略 I：強化身分驗證與憑證

**多因子驗證（MFA）**
這是對抗撞庫與暴力破解唯一有效的阻斷機制。
避免依賴知識型問答（KBA）。

**現代化密碼策略（NIST 800-63B）**

- **Do Not Rotate**：
  停止強制定期更換密碼（除非懷疑洩漏）。
- **Complexity**：
  鼓勵長通關密語（Memorized Secrets），
  而非過度複雜的字元規則。
- **Tooling**：
  鼓勵使用者使用密碼管理員（Password Managers）。

**主動檢測與攔截（Active Detection）**

- **Weak Check**：
  對照「Top 10,000 worst passwords」檢查新密碼。
- **Breach Validation**：
  使用 HaveIBeenPwned 等清單驗證憑證安全性。
- **Throttling**：
  實施速率限制（Rate Limiting）與延遲機制
  以阻斷自動化腳本。

SYSTEM DIAGNOSTICS v1.0

DATE: 2024-05-20

# A07:2025 Authentication Failures

## 防禦策略 II：會話管理與架構設計

### 會化衛生(Session Hygiene)

**ID 產出**：使用伺服器端內建的高熵值產生器。
**儲存安全**：禁止在 URL 傳遞，必須使用 Secure Cookie（HttpOnly、Secure）。
**生命週期**：登出、閒置或絕對超時後，立即作廢 Session ID。

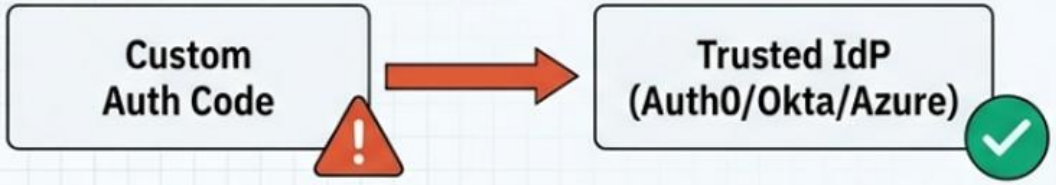
### 防枚舉設計（Anti-Enumeration）

統一回應訊息（Generic Error Messages）

```
return "Invalid username or password";
```

無論是帳號不存在或密碼錯誤，皆回傳一致訊息，防止攻擊者刺探有效帳號。

### 架構決策（Architectural Decisions）




不要重複造輪子。盡可能轉移風險，
採用經過驗證的 IdP 系統，而非自行構建身分驗證邏輯。

# A07:2025 Authentication Failures

## 開發者實戰檢查清單（Developer Checklist）

### YES – Do This

- 實施多因子驗證（MFA）
- 驗證密碼是否在洩漏清單中（HaveIBeenPwned）
- 登入失敗時使用通用錯誤訊息
- 使用高熵值 Session ID 並存在 Secure Cookie 中
- 遵循 NIST 800-63B（取消強制定期更換密碼）

### NO – Avoid This

- 使用預設帳號或密碼（Default Creds）
- 允許弱密碼（如 'Password1'）
- 在 URL 中傳遞 Session ID
- 登出時僅清除前端 Cookie（必須作廢伺服器端 Session）
- 使用「知識型問答」（KBA）進行密碼重置

# A07:2025 Authentication Failures

## 延伸閱讀與 CWE 對照

| 主要參考文件 (Key References) | 重點 CWE 列表 (Mapped CWEs) |
|---|---|
| • OWASP Authentication Cheat Sheet | CWE-259: Use of Hard-coded Password |
| • OWASP Secure Coding Practices | CWE-798: Use of Hard-coded Credentials |
| • NIST 800-63b (Digital Identity Guidelines) | CWE-287: Improper Authentication |
| | CWE-384: Session Fixation |
| | CWE-613: Insufficient Session Expiration |
| | CWE-308: Use of Single-factor Authentication |
| | CWE-640: Weak Password Recovery Mechanism |

*“身分驗證極其複雜。請依賴標準框架，避免自行撰寫核心驗證代碼。”*

# A07:2025 Authentication Failures

## A07: Identification and Authentication Failures (身分驗證失效)

### 常見攻擊案例與防禦邏輯解析

TARGET AUDIENCE: R&D / DEVELOPER TEAM

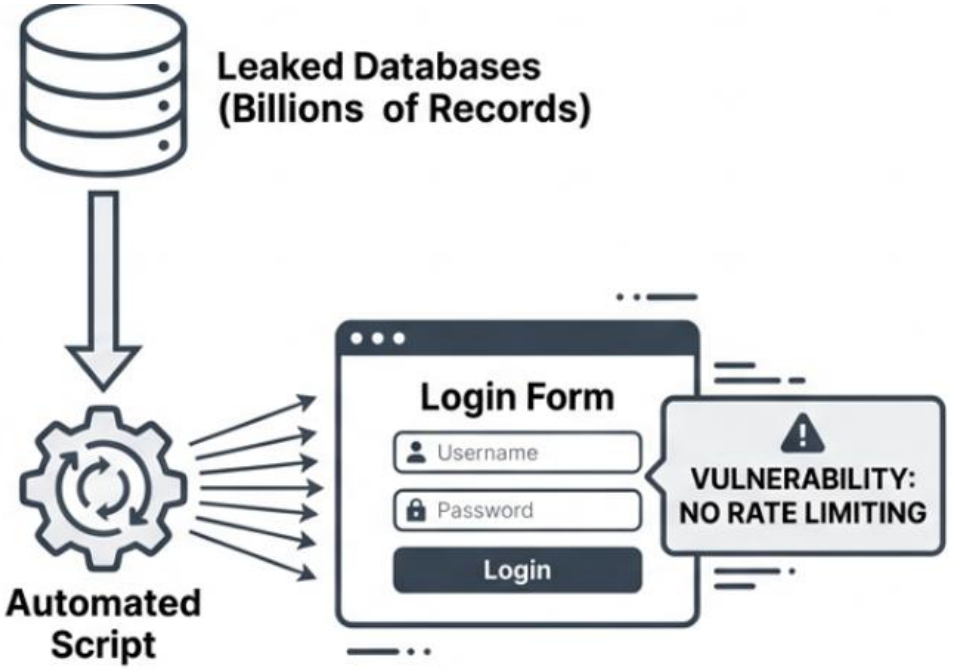
基於 OWASP Top 10 A07 標準
重點：帳號枚舉、暴力破解與邏輯缺陷

# A07:2025 Authentication Failures

## 攻擊者的資料來源：憑證填充與暴力破解

**資料收集（Data Collection）**
攻擊者利用 **Have I Been Pwned** 等資料庫，
獲取已洩漏的使用者名稱與密碼。
這些清單包含**數十億組真實憑證**，
可供下載並用於自動化攻擊

**攻擊手法（Attack Methods）**
**Credential Stuffing（憑證填充）**：
使用洩漏的帳號密碼清單，嘗試登入其他網站。
**Brute Force（暴力破解）**：
使用軟體每秒嘗試數千種密碼組合。

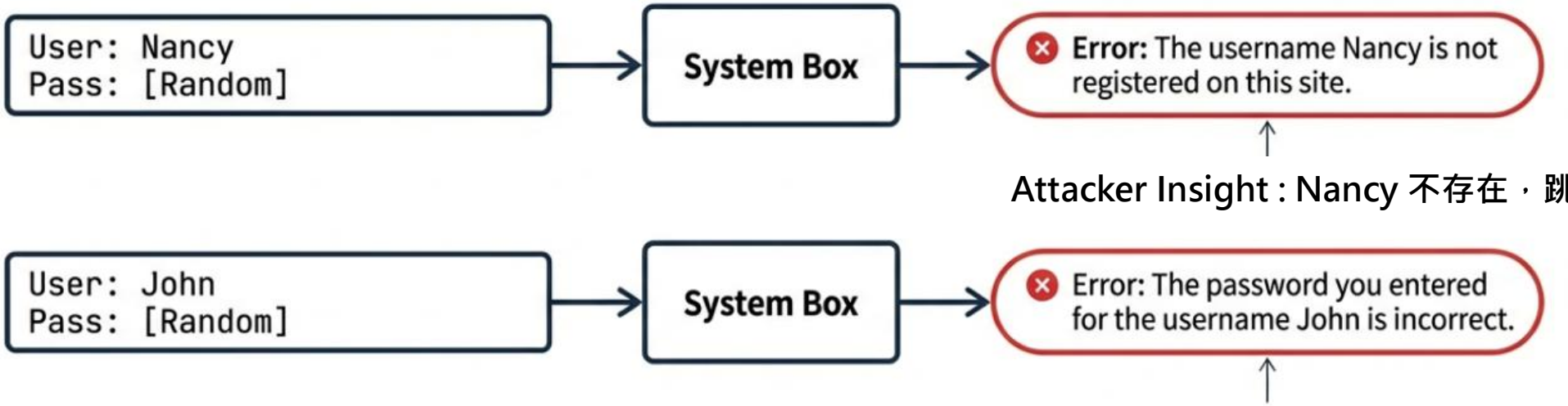



**如果應用程式缺乏「速率限制」或「逾時機制」，將成為攻擊者的練兵場。**

# A07:2025 Authentication Failures

## 案例演示 A：登入邏輯中的帳號枚舉

Flowchart of Failure




Attacker Insight : Nancy 不存在，跳過。

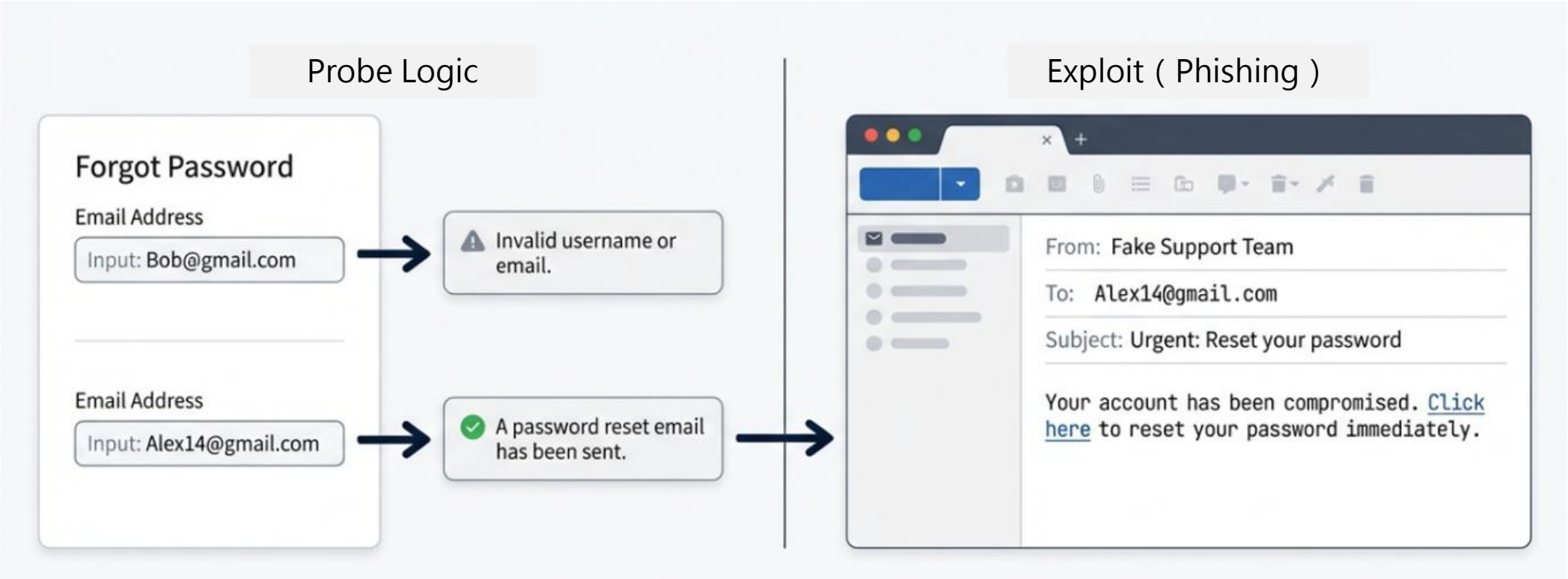
Attacker Insight : John 是真實用戶！（John exists）

Impact
錯誤訊息洩漏了用戶存在的事實，
攻擊者現在可以針對 John 進行集中式暴力破解，
大幅縮小攻擊範圍。

# A07:2025 Authentication Failures

## 案例演示 B：密碼重設功能的資訊洩漏

利用「忘記密碼」功能進行探測

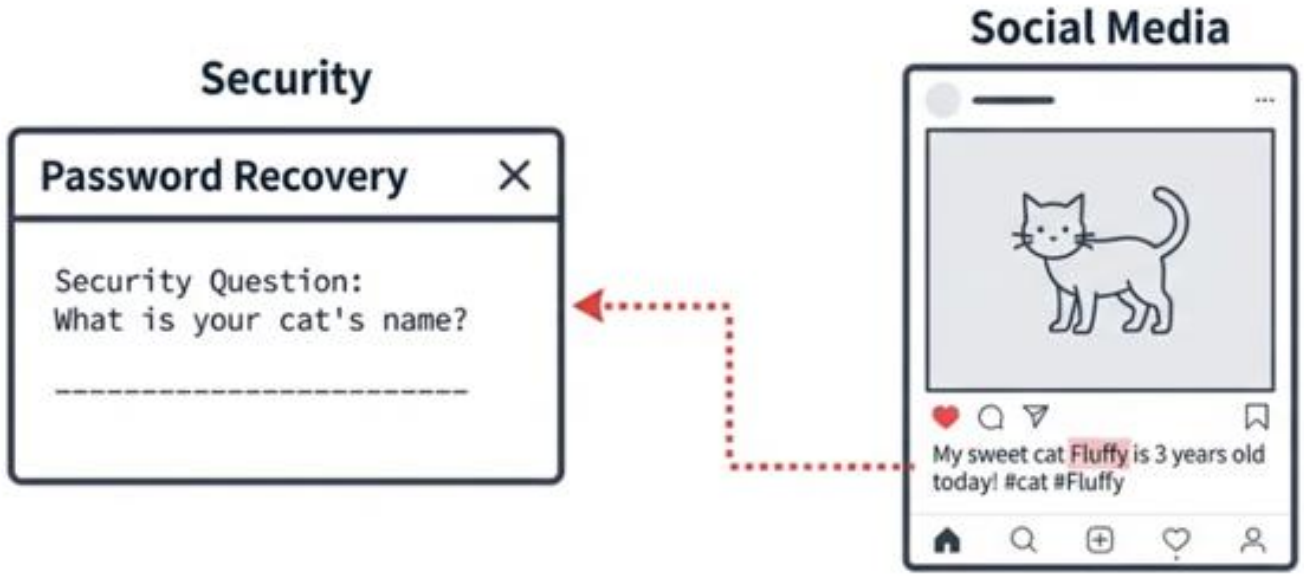



**The Exploit (Phishing Risk)**

攻擊者確認 **Alex14 為註冊用戶** 後，偽裝成該網站發送「釣魚郵件」，謊稱帳號受駭，誘騙 Alex 輸入憑證。

# A07:2025 Authentication Failures

## 過時的驗證機制：知識型問答（KBA）




**Why It Fails（OSINT Attack）**
攻擊者利用開源情報（OSINT）鎖定目標社交媒體。
靜態問題的答案等同於**極弱的密碼**，且**無法安全地重置**。
**結論：** 避免使用知識型問答作為**唯一**的帳號恢復手段。

# A07:2025 Authentication Failures

## 防禦實作 I：回應邏輯與模糊化

**核心原則：** 保持模糊，讓攻擊者無法區分

**登入修正（Login Refactoring）**

x Bad Pattern

"User not found"

"Wrong password"

v Good Pattern

**"Invalid username or password"**

（無效的使用者名稱或密碼）

**Result：**

攻擊者無法得知是帳號錯還是密碼錯。

**重設流程修正（Password Reset Refactoring）**

x Bad Pattern

"Email sent" **vs** "Email not found"

v Good Pattern

**"If the email address exists on this website, a password reset link will be sent to it."**

（如果此 Email 存在於本站，重設連結將會發送至該信箱）

**Key Logic：**

使用 **IF 語句**，**無論 Email 是否存在**，

前端顯示的訊息**必須完全一致**。

# A07:2025 Authentication Failures

OWASP TOP 10 2025

## A07:2025 Identification & Authentication Failures

### 拒絕帳號枚舉與時序攻擊：從錯誤的登入驗證談起

💡 觀念釐清：常誤判為 A01 Broken Access Control，但此為登入回應差異導致的資訊洩露。

# A07:2025 Authentication Failures

## 程式碼審查：這段登入驗證哪裡有問題？

```php
private function authenticate(string $email, string $password) {
    //    問題點 1: 根據帳號是否存在，回傳不同訊息
    if (!isset($this->mockerUsers[$email])) {
        http_response_code(401);
        echo "User not found";
        exit;
    }

    //    問題點 2: 只有密碼錯誤時才延遲 (時序攻擊漏洞)
    if ($this->mockerUsers[$email] !== $password) {
        sleep(1);
        http_response_code(401);
        echo "Incorrect password";
        exit;
    }
    // ... login success logic
}
```

訊息不一致 (Inconsistent Messages)

條件式延遲 (Conditional Delay)

明文比對 (Plain Text Comparison)

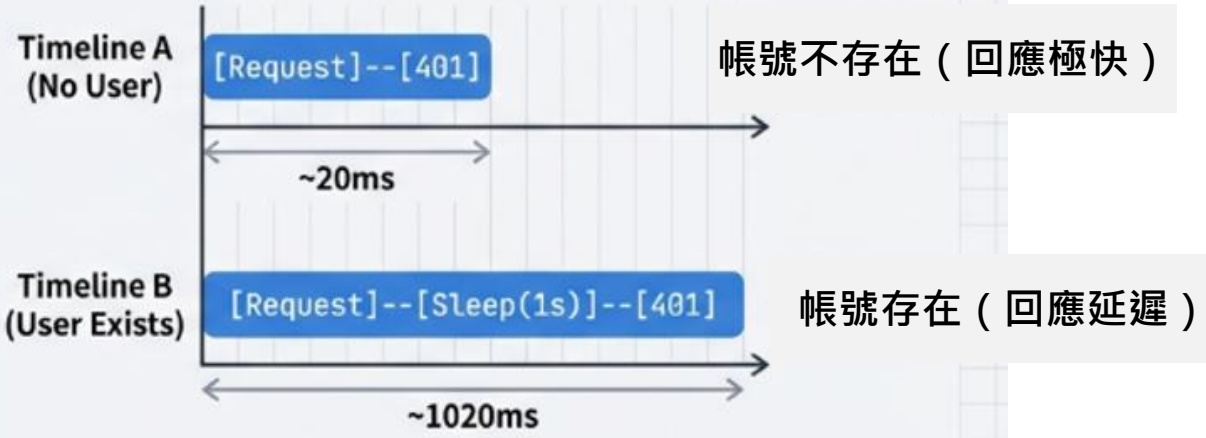
# A07:2025 Authentication Failures

## 漏洞診斷：攻擊者看到了什麼？

### 帳號枚舉（User Enumeration）

攻擊者可直接篩選有效帳號清單。

### 時序攻擊（Timing Attack）




即使訊息隱藏，時間差異也會洩密。

另外：明文比對密碼極度危險，真實系統必須使用 password_hash()。

# A07:2025 Authentication Failures

- **修正方案：統一錯誤訊息與時間行為**

```php
private function authenticate(string $email, string $password): array {
    //  修正 1: 加入隨機延遲，模糊化時間差異
    usleep(random_int(100_000, 300_000));

    //   修正 2: 統一錯誤訊息，不區分帳號或密碼錯
    if (!isset($this->mockerUsers[$email]) ||
        $this->mockerUsers[$email] !== $password) { // 註: 真實場景請用 password_verify

        http_response_code(401);
        echo json_encode(['error' => 'Invalid login credentials.']);
        exit;
    }

    return ['message' => 'Login successful', 'token' => '...'];
}
```

**防時序攻擊**
確保失敗路徑耗時隨機化（100ms–300ms）
無法由時間推斷邏輯

**防帳號枚舉**
無論原因為何，一律回傳
**Invalid login credentials**

# A07:2025 Authentication Failures

## 核心總結與最佳實踐

**統一回應（Unified Response）**

- 登入失敗時，**訊息內容必須完全一致**。

**模糊時間（Timing Obfuscation）**

- 加入隨機延遲，
- 確保失敗路徑的執行時間**無法區分帳號狀態**。

**正確分類（Correct Classification）**

- 這是 **A07 登入回應差異漏洞**，
- **而非 A01 權限控制問題**。

**企業級開發建議**

- 避免在業務邏輯中散落 echo 與 exit。
- 建議做法：

  拋出 Exception（如 throw new AuthException）

  由**全域 Exception Handler** 統一格式化 JSON 回應

- 好處：

  程式碼整潔

  回應一致

  安全策略集中管理

# A08:2025 Software or Data Integrity Failures

OWASP TOP 10

**A08:2025**

**軟體與資料完整性失效**

Software or Data Integrity Failures

針對開發人員與 R&D 團隊的技術診斷與防禦指南

核心概念：當我們信任未經驗證的來源，代碼就會成為攻擊載體。

# A08:2025 Software or Data Integrity Failures

## 信任邊界的崩潰

*Definition: The Collapse of Trust Boundaries*

此類別聚焦於在未驗證完整性的情況下，
對軟體更新、關鍵資料與 CI/CD 管道做出的假設。

2025 更新：
名稱從「Software and Data」微調為「Software or Data」，
強調單一失效即可導致系統崩潰

- **盲目信任**

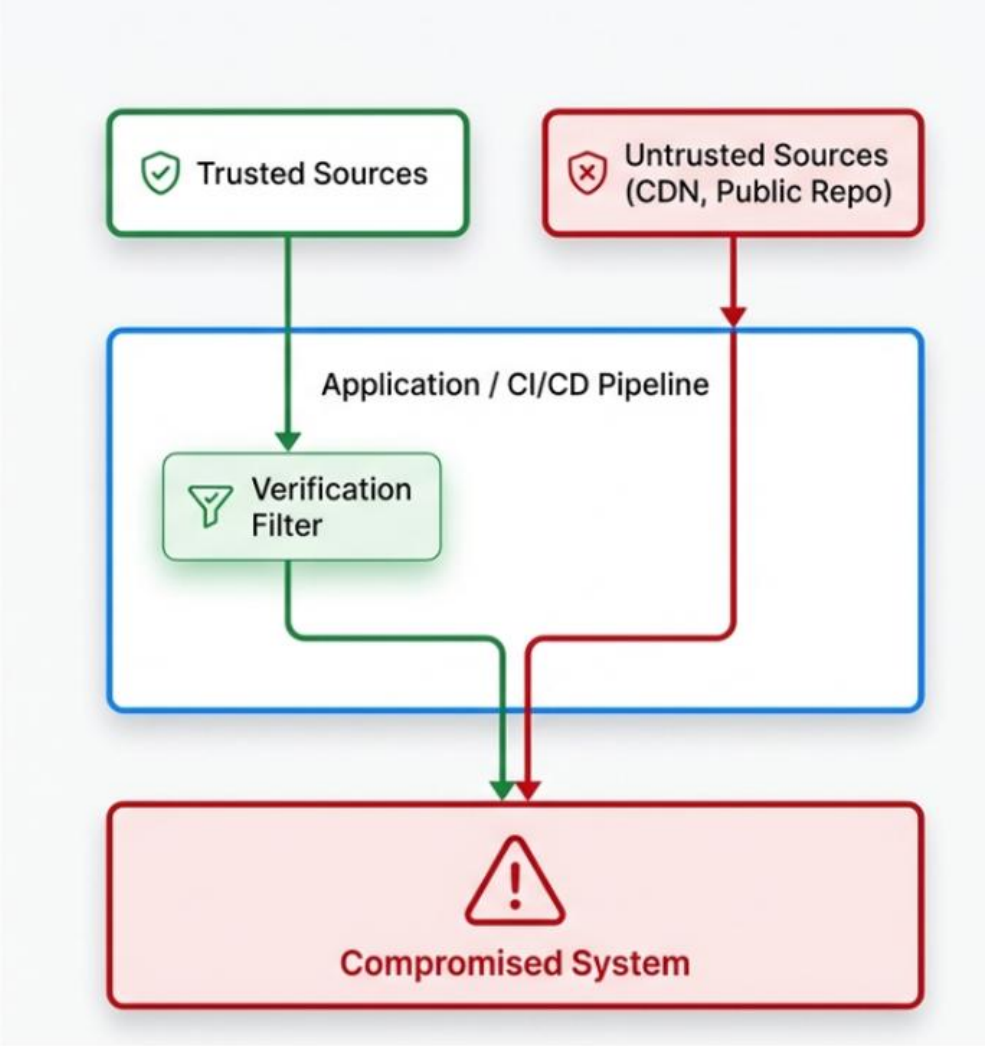
應用程式依賴來自不信任來源（如 CDN、公開 Repositories）的插件或模組。

- **自動化風險（Automation Risk）**

自動更新（Auto-update）在下載與套用更新時，
未進行充分的完整性驗證。

- **基礎設施漏洞**

CI/CD 管道在沒有完整性檢查的情況下
提取或使用程式碼與 artifacts。



# A08:2025 Software or Data Integrity Failures

## 技術機制與 CWE 解析 *(Technical Mechanics & CWE Analysis)*

CWE-502 — 不安全的反序列化

- 攻擊者利用惡意序列化物件（Serialized Objects）取代合法資料。
- 若應用程式在**未驗證的情況下反序列化**，將導致 **RCE**

CWE-829 — 包含不信任控制範圍的功能

- 從第三方來源（如 **npm**、**Maven**、**CDN**）提取函式庫或模組。
- 卻**未驗證其來源或完整性**。

CI/CD Pipeline — 管道風險

- 管道缺乏存取控制與配置隔離。
- 攻擊者可注入惡意代碼，並**自動部署至生產環境**。

# A08:2025 Software or Data Integrity Failures

## 常見攻擊場景（Common Attack Scenarios）

**Scenario 1：套件來源欺騙**

```
$ npm install angular-cli-core
> Warning: Unverified maintainer...
> Installing malicious payload...
```

**開發者誤下載 Typosquatting 套件，引入惡意代碼。**

**Scenario 2：不安全的反序列化**

```
String userData = request.getParameter("state");
// VULNERABILITY: Deserializing without check
ObjectInputStream ois = new ObjectInputStream(...)
Object state = ois.readObject();
```

Spring / React 應用中直接反序列化 **User State**，導致 **RCE**。

**Scenario 3：韌體更新未驗證**

**IoT 設備下載更新未檢查簽章，被植入後門。**

# A08:2025 Software or Data Integrity Failures

## 防禦與緩解策略（Defense & Mitigation Strategies）

**1. 數位簽章**
確保所有軟體更新、Artifacts 與數據都經過簽章驗證。
驗證來源是否符合預期，且數據未被竄改。

**2. 供應鏈安全**
使用 OWASP Dependency Check 掃描漏洞。
建立內部的受信任存儲庫（Nexus / Artifactory），避免直接使用公開開源。

**3. 強化 CI/CD**
實施代碼審查（Code Review）與配置變更審核。
確保管道內的職責分離（Segregation of Duties）。

**4. 數據完整性**
對於來自不信任客戶端的序列化數據，
必須進行完整性檢查或加密驗證後方可使用。

# A08:2025 Software or Data Integrity Failures

## 開發者檢查清單（Developer Checklist）

完整性是信任的基礎（Integrity is the foundation of trust）

- [ ] 我們的軟體更新是否使用了數位簽章並在安裝前進行驗證？
- [ ] 我們的 CI/CD 管道是否有隔離與審查機制？
- [ ] 我們是否只從受信任的內部存儲庫（Trusted Repositories）拉取代碼與依賴項？
- [ ] 接收序列化數據時，是否進行了完整性檢查以防止竄改？

不要假設信任，永遠驗證。（Don't assume trust; Always verify.）

# A08:2025 Software or Data Integrity Failures

OWASP TOP 10 SECURITY DEEP DIVE

## A08:2025 Software or Data Integrity Failures

實戰解析：從 PHP 反序列化漏洞到安全架構

本簡報將深入探討 CWE-502（Deserialization of Untrusted Data）。
我們將剖析一個看似正常的 PHP 類別如何因缺乏完整性驗證（Integrity Check），演變成嚴重的安全漏洞

目標 Goal
理解漏洞成因（Root Cause）
分析 Gadget Chain 攻擊原理
掌握從 Patch 到 Refactor 的修正策略

受眾 Audience
後端開發者
資安工程師
系統架構師

# A08:2025 Software or Data Integrity Failures

## 錯誤示範：未經驗證的信任（The Mistake: Unverified Trust）

```php
public function run(): void {
    header("Content-Type: application/json");

    // 接收輸入
    $data = file_get_contents("php://input");
    $decoded = json_decode($data, true);

    //  VULNERABILITY: User input is trusted blindly
    $payload = base64_decode($decoded["payload"]); ⚠
    $object = unserialize($payload);

    echo json_encode(["message" => "Object deserialized"]);
}
```

**CWE-502 違規核心**
程式碼直接將使用者可控的輸入（$decoded['payload']）傳入 unserialize()，
這打破了信任邊界。

**常見誤解**
base64_decode 僅是編碼轉換，
並非加密或安全驗證措施。
任何人都可以解碼、修改並重新編碼 Payload。

**A08 風險脈絡**
缺乏 HMAC 或數位簽章來驗證資料完整性。
攻擊者可以隨意竄改 Payload，
注入惡意物件（Object Injection）。

# A08:2025 Software or Data Integrity Failures

## 潛在危機：被濫用的組件（The Consequence: The Gadget）

```php
class Cache {
    protected string $cacheFile;

    public function __destruct() {
        // PHP 自動執行此方法
        unlink($this->cacheFile);
    }
}
```

危險操作：刪除檔案

危險操作：刪除檔案

**什麼是 Gadget？**
這個 Cache 類別本身是用來刪除暫存檔的，
但在反序列化攻擊中，它是攻擊者的武器。

**關鍵教訓**
永遠不要讓使用者決定「什麼時候」
以及「用什麼參數」來執行你的 Magic Methods。

**Attacker Input**
**Serialized Object**
$cacheFile = '/etc/passwd'

→

**Vulnerability**
Slide 2 Code
unserialize($payload)

→

**Trigger**
Script Ends
__destruct() runs automatically

→

**Arbitrary File Deletion**
unlink('/etc/passwd')

# A08:2025 Software or Data Integrity Failures

## 加入完整性驗證（Attempted Fix: Integrity Verification）

```php
// 1. 驗證 HMAC (Hash-based Message Authentication Code)
// 確保資料未被竄改 (Integrity Check)
if (!hash_equals(hash_hmac('sha256', $payload, $secretKey), $hmac)) {
    throw new HttpException("Invalid hmac sign");
}

// 2. 限制允許的類別 (Allowed Classes)
// 減少攻擊面 (Attack Surface Reduction)
$allowedClasses = ['App\helpers\SafeClass'];
$object = @unserialize($payload, ['allowed_classes' => $allowedClasses]);
```

**驗證完整性（Integrity Check）**
**確保資料未被竄改，**
**只有持有 secretKey 才能產生有效 Payload。**

**減少攻擊面（Attack Surface Reduction）**
限制允許的類別，
降低惡意物件注入風險。

**建立信任邊界（Trust Boundary）**
**利用 hash_hmac 確保資料在傳輸過程中未被竄改。**
**只有持有 $secretKey 的人才能產生有效 Payload。**
**這符合 A08 的核心要求：驗證完整性。**

**潛在限制（Potential Limitations）**
雖然加上了防護，但 **PHP 官方文件仍建議不要對不可信資料使用 unserialize()**。
底層解析邏輯複雜，可能隱藏未知的記憶體問題，且維護困難。

# A08:2025 Software or Data Integrity Failures

## 最佳實踐：架構重構（Best Practice: Architectural Refactoring）

### Transformation

JSON Architecture

```
//    移除 serialize/unserialize 機制
//    改用 JSON 進行資料交換

$message = (string)$decoded['message'];

// 手動實例化 (Explicit Instantiation)
// 由開發者決定建立什麼物件，而非由資料決定
$object = new \App\helpers\SafeClass($message);

echo json_encode(['message' => 'OK', 'object' => $object]);
```

**資料格式轉換（Data Format）：**
使用 JSON 取代 Serialized String。JSON 只是數據，不會自動觸發 PHP 的 Magic Methods（如 __destruct）。

**控制權回歸（Control Flow）**
開發者明確定義 new SafeClass()，
攻擊者無法透過改變數據結構來強迫系統建立 Cache 或其他危險物件。

結論（Verdict）：與其修補 unserialize 的漏洞，不如從架構上移除這個危險機制。

# A08:2025 Software or Data Integrity Failures

## 重點回顧與防禦清單（Key Takeaways & Checklist）

**防禦策略（Defense Strategy）**

**拒絕不安全的反序列化：**
避免在使用者輸入上使用 unserialize()。

**使用標準數據格式：**
優先選擇 JSON 或 XML 進行資料交換。

**完整性驗證（Integrity Check）：**
若必須傳遞敏感狀態，務必使用數位簽章（Digital Signatures）
或 HMAC 驗證資料來源。

**審計 Magic Methods：**
檢查程式碼中的 __destruct、__wakeup、__toString，
確保它們不會執行危險操作。

參考資料（References）


OWASP Top 10: 2025 A08 (Software or Data Integrity Failures)
CWE-502: Deserialization of Untrusted Data
OWASP Cheat Sheet: Deserialization
PHP Manual: Unserialize Security Warning